

1. 프로젝트 개요

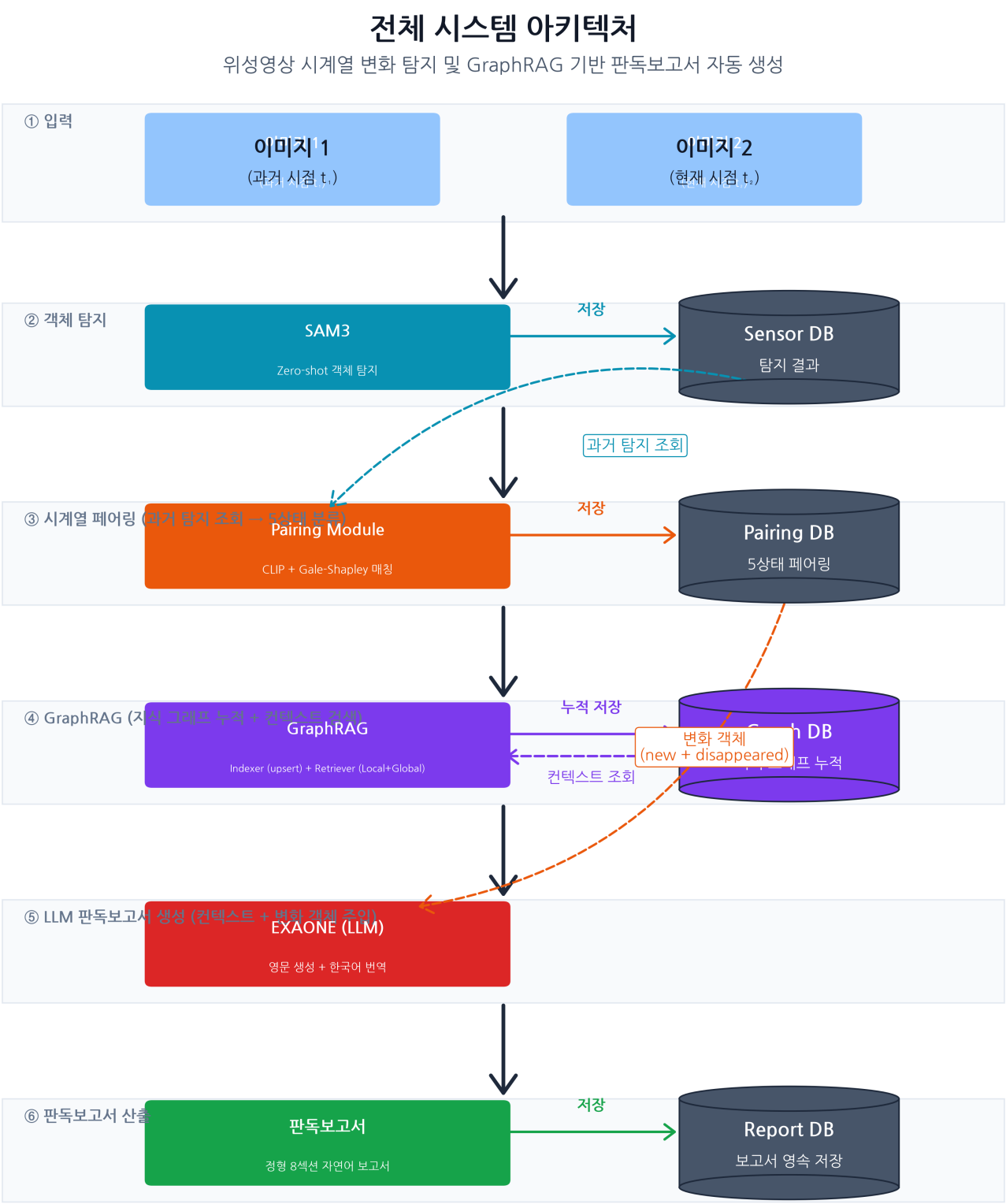
본 프로젝트(MSIS, Multi-Source Intelligent System)는 위성·드론 시계열 영상 한 쌍을 입력받아 변화를 자동 탐지하고, 그 결과를 정형 판독보고서로 자율 생성하는 end-to-end 통합 시스템이다. 종래의 픽셀 단위 패치 격자 비교 방식이 객체 이동을 추적하지 못하고 이진 판정에 그치며 시계열 누적 패턴을 인식하지 못하는 한계, 그리고 LLM 기반 보고서 생성 시 시계열 컨텍스트 부재와 환각 위험 문제를 함께 해결하는 것을 목표로 한다.

시스템은 객체 인스턴스 단위 페어링 기반 변화 탐지와 결정론적 GraphRAG 이력 누적을 결합한 뒤, LangGraph 기반 critic 노드를 갖춘 AI Agent 구조로 판독보고서를 자율 생성한다. 각 파이프라인 단계(탐지 · 페어링 · 그래프 인덱싱 · 보고서 생성 · 번역)의 산출물은 critic 노드에서 자동으로 검증되며, 검증 실패 시 해당 단계로 자동 재실행되거나 사람에게 이관되는 자기 검증 구조를 갖는다. 각 단계의 산출물은 별도 데이터베이스에 세션 식별자와 함께 저장되어 결과의 재현과 검증이 가능하다.

핵심 기여점

- 객체 인스턴스 단위 이원 페어링 — 고정형(위경도 근접 + CLIP)과 이동형($N \times M$ 유사도 행렬 + Gale-Shapley 안정 매칭)으로 분리 처리, 다상태(6종) 정밀 분류
- 결정론적 지식 그래프 누적 — 위경도 소수점 2자리 반올림(약 1km 격자) 결정론 노드 키, AI 호출 없는 upsert, Louvain 커뮤니티 탐지로 반복 관측 패턴 자동 발견
- LangGraph critic 기반 자율 판독보고서 생성 — Local + Global 압축 컨텍스트를 LLM에 주입, 영문 생성 → 한국어 번역 각 단계에 critic 노드를 배치해 환각·형식 오류 자동 검증 및 재실행
- 세션 단위 재현 가능 4-DB 아키텍처 — Sensor/Pairing/Graph/Report DB를 물리적으로 분리, session_id 기반 HITL 재처리 지원

2. 전체 시스템 아키텍처



각 DB는 세션 식별자(session_id)로 묶여 HITL 재처리 시 자동 재계산됩니다.

[도 1] 전체 파이프라인 흐름도

2. 전체 시스템 아키텍처 (설명)

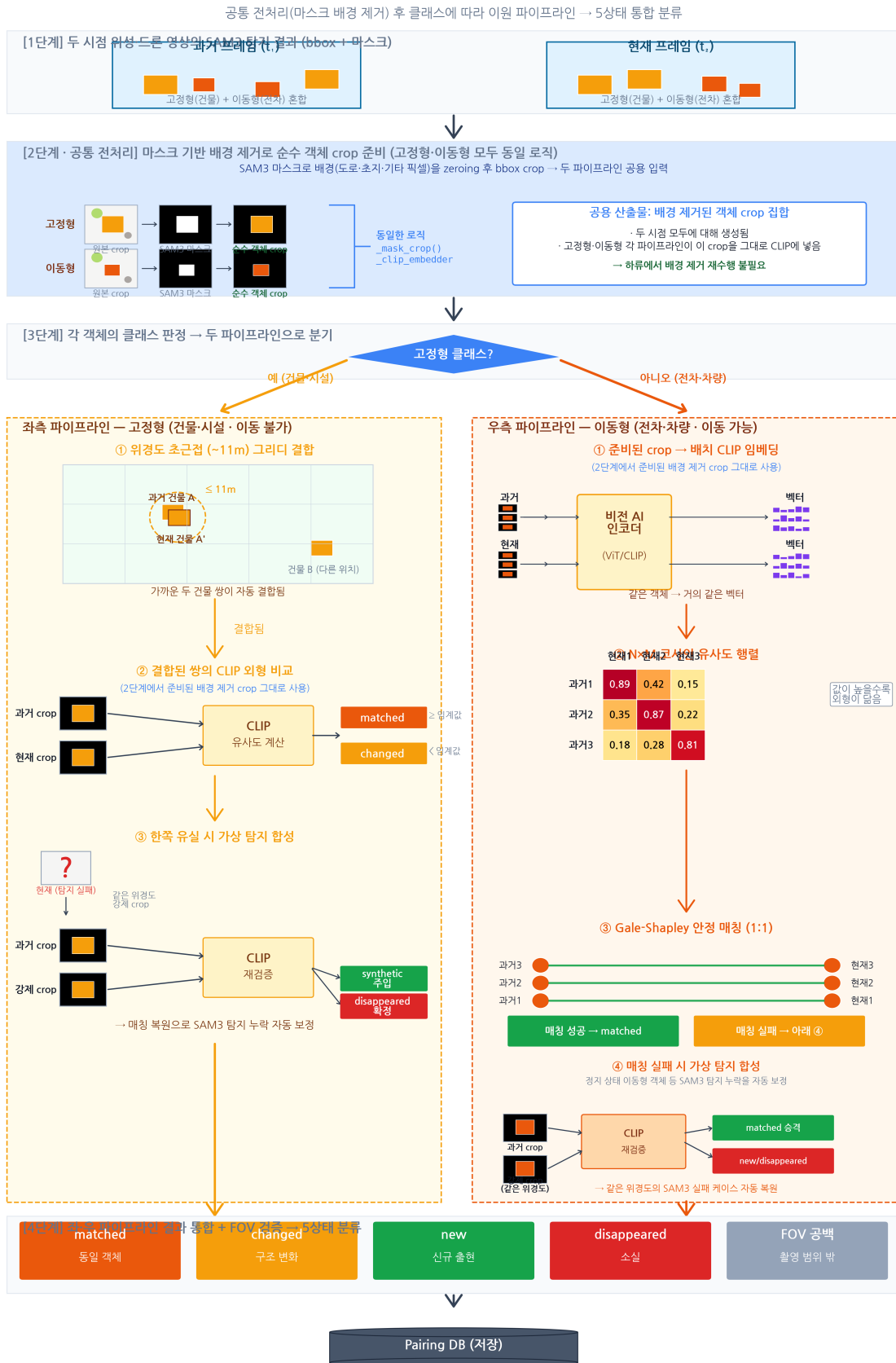
도 1은 두 시점 위성·드론 영상 한 쌍에서 시작해 최종 판독보고서가 나오기까지의 전체 흐름을 보여준다. 시스템의 핵심은 탐지 → 페어링 → 지식 누적 → 보고서 생성의 4단계 파이프라인이다.

각 단계는 LangGraph 노드로 모델링되고, 각 단계 뒤에는 critic 노드가 배치되어 산출물의 품질을 자동 검증한다. 검증을 통과하면 다음 단계로 진행하고, 실패하면 같은 노드로 재실행되거나 임계값 조정 후 재시도되며, 반복 실패 시 HITL(Human-in-the-Loop)로 이관되는 자기 검증 구조이다.

각 단계의 산출물은 별도 데이터베이스(Sensor / Pairing / Graph / Report)에 저장되어 결과를 재현하고 검증할 수 있으며, 특정 단계의 결과가 사용자에게 의해 수정되면 후속 단계가 자동 재계산되는 구조를 갖는다.

3. 객체 페어링 변화 탐지 상세 (이원 처리)

도 2. 객체 페어링 변화 탐지 상세



[도 2] 페어링 모듈 — 공통 전처리 + 이원 파이프라인

3. 객체 페어링 변화 탐지 상세 (설명)

도 2는 두 시점 영상의 객체를 어떻게 짝지어 변화를 잡아내는지를 4단계 흐름으로 보여준다: (1) SAM3 탐지 입력 → (2) 마스크 기반 배경 제거 공통 전처리 → (3) 클래스 판정 후 이원 파이프라인 → (4) 결과 통합 및 다상태 분류. 핵심 아이디어는 객체의 물리적 성질에 따라 페어링 방식을 완전히 달리하여, 이동 가능성 여부에서 오는 알고리즘 특성 차이를 파이프라인 수준에서 흡수한다는 점이다.

(1) SAM3 탐지 입력

과거 시점(t_1)과 현재 시점(t_2) 두 영상 각각에 SAM3 (Segment Anything Model 3)의 zero-shot 탐지를 수행한다. 사용자는 원하는 객체 클래스명을 자연어 프롬프트(예: "military tank", "military building")로 지정하고, SAM3는 각 객체 인스턴스에 대해 픽셀 단위 bbox, 세그멘테이션 마스크, confidence 값을 반환한다. 반환된 픽셀 좌표는 이미지의 지리 bounding box(lat_min/max, lon_min/max)를 이용해 선형 보간으로 위경도(lat, lon)로 변환되어 Sensor DB에 저장된다.

(2) 마스크 기반 배경 제거 (공통 전처리)

두 시점 모두에 대해 SAM3 마스크로 배경 픽셀(도로·초지·기타 무관 요소)을 zeroing한 뒤 bbox 영역만 crop 하여 **순수 객체 crop**을 생성한다. 이 crop은 이후 고정형·이동형 두 파이프라인이 **공용**으로 사용하며, 하류에서 배경 제거를 재수행할 필요가 없다. 배경이 제거된 crop을 CLIP에 입력함으로써 주변 지형 노이즈에 강인한 순수 객체 외형 임베딩을 얻을 수 있어, 회전·조명 변화·촬영 각도가 다른 두 시점 간 유사도 계산의 신뢰도가 확보된다.

(3-a) 고정형 객체 처리 — 지리 근접 + CLIP 재검증

건물·시설과 같은 **고정형**(_STATIC_CLASSES 로 코드에서 지정) 객체는 물리적으로 위치가 고정되어 있으므로, 두 시점 사이에도 위경도가 거의 변하지 않는다. 이를 활용해 **위경도 초근접(약 11m, 0.0001° 단위) 그리디 결합**을 먼저 수행하여 공간적으로 가까운 과거·현재 객체 쌍을 후보로 묶는다.

결합된 각 쌍에 대해 **_clip_similarity_crops()** 함수가 두 crop의 CLIP 임베딩 코사인 유사도를 계산한다. 유사도가 임계값(_STATIC_SIM_THRESHOLD) 이상이면 **matched**(구조 변화 없음), 미만이면 **changed**(신축·증축·철거·피해 등 구조적 변화)로 판정한다. 위경도가 거의 같아도 CLIP 외형이 달라졌다는 것은 시설물의 구조적 변화를 강하게 시사하므로, 단순 위치 비교로는 감지할 수 없는 '같은 자리, 다른 모습' 케이스를 잡아낼 수 있다.

SAM3가 한쪽 시점에서 탐지에 실패한 경우에는 **가상 탐지 합성(synthetic detection injection)** 메커니즘이 동작한다. 유실된 시점의 이미지에서 반대편 시점의 동일 위경도 영역을 강제로 crop 하여 CLIP 재검증을 수행하고, 유사도가 임계값을 넘으면 synthetic detection을 주입하여 matched로 복원하고, 그렇지 않으면 disappeared로 확정한다. 이 절차는 SAM3 탐지 누락을 자동 보정하여 이후 GraphRAG 누적 통계의 왜곡을 방지한다.

3. 객체 페어링 변화 탐지 상세 (설명 계속)

(3-b) 이동형 객체 처리 — CLIP 유사도 행렬 + Gale-Shapley 안정 매칭

전차·차량과 같은 **이동형** 객체는 두 시점 사이 위경도가 크게 달라질 수 있으므로 위치 근접만으로는 매칭이 불가능하다. 대신 각 시점의 crop들을 배치 단위로 CLIP 인코더에 입력해 N개(과거)·M개(현재)의 L2 정규화 임베딩 벡터를 얻고, 이를 **단일 GEMM 연산**(`cur_embeds @ past_embeds.T`)으로 $N \times M$ 코사인 유사도 행렬을 계산한다. 이 행렬의 각 원소는 두 객체 간 외형적 유사도(0 ~ 1)를 나타내며, 값이 임계값(`SIMILARITY_MATCH_THRESHOLD`, 예: 0.5) 이상인 조합만 매칭 후보에 포함된다. 동일 클래스가 아닌 조합(예: tank ↔ APC)은 사전 필터로 제외된다. 각 후보의 최종 점수는 **$0.8 \times \text{CLIP 유사도} + 0.2 \times \text{크기 유사도}$** 로 가중 합산되며, 크기 유사도가 반영되어 잘못된 스케일 매칭을 억제한다.

이 후보 리스트에 **Gale-Shapley 안정 매칭(deferred-acceptance)**이 적용된다. 각 현재 객체(c_i)는 자기 선호도 리스트(점수 내림차순)의 최상위 과거 객체(p_i)에게 순차적으로 제안한다. p_i 가 비어 있으면 잠정 매칭이 성립하고, 이미 다른 c_j 와 매칭된 경우엔 두 c_i 의 점수를 비교하여 더 높은 쪽이 p_i 를 획득하고 밀린 c_j 는 자기의 다음 후보로 재도전한다. 각 c_i 의 후보 포인터가 단조 증가하므로 유한 스텝 안에 반드시 종료되며, 결과 매칭에는 **서로 바꿔치기 하고 싶은 짝이 남지 않는다**는 안정성이 수학적으로 보장된다. 이 성질 덕분에 그리디 매칭에서 흔히 발생하는 **중복 매칭·교차 매칭**이 원천 차단되어, 밀집 지역의 다수 이동형 객체 페어링 신뢰도가 크게 향상된다.

매칭에 성공한 쌍은 `matched`로 확정되고, 매칭에 실패한(임계값 미달 또는 후보 소진) 현재 객체는 `new`, 매칭되지 않은 과거 객체는 `disappeared`로 **임시** 표시된다. 이 임시 결과는 다음의 재검증 단계를 한 번 더 거친다.

(3-c) 이동형 매칭 실패 재검증 — 가상 탐지 합성

이동형이라 하더라도 전차·차량이 **정지 상태로 주둔**하는 경우가 흔한데, 이 경우 SAM3가 한쪽 시점에서 탐지에 실패하면 Gale-Shapley 후보에서 빠져 `new·disappeared`로 잘못 분류될 수 있다. 이를 보정하기 위해 **고정형과 동일한 `_static_cross_check()` 재검증**이 이동형에도 대칭적으로 적용된다.

구체적으로, 임시로 `new`가 된 이동형 객체는 과거 이미지에서 **현재 탐지의 위경도 위치**를 강제 crop 하고, 임시 `disappeared`가 된 객체는 현재 이미지에서 **과거 탐지의 위경도 위치**를 강제 crop 하여 원본 crop과의 CLIP 코사인 유사도를 계산한다. 유사도가 `_STATIC_SIM_THRESHOLD` 이상이면 "같은 자리에 여전히 있으나 SAM3가 놓친 경우"로 판정하여 **합성 DetectionRecord를 유실된 프레임에 주입**하고 `matched`로 승격시킨다. 임계값 미만이면 실제로 등장/사라진 것으로 최종 확정된다.

이 재검증 단계 덕분에 SAM3의 false negative(정지 상태 이동 자산에 대한 탐지 누락)가 GraphRAG 누적 통계와 판독보고서 서술로 전파되는 것을 자동으로 차단할 수 있다. 결과적으로 고정형·이동형 두 파이프라인 모두 **1차 매칭 → 실패분에 대한 CLIP 재검증 → 최종 확정**이라는 대칭 구조를 가진다.

(4) 결과 통합 및 다상태 분류

고정형·이동형 두 파이프라인의 페어링 결과와 각 이미지의 FOV(field-of-view, 촬영 범위) 검증 결과를 통합하여 최종적으로 6개의 상태 중 하나로 분류한다:

- **matched** — 두 시점 모두에 존재하며 유사도 임계값 이상
- **changed** — 같은 위치 고정 시설이지만 CLIP 유사도가 임계값 미만 (구조 변화)
- **new** — 현재 시점에만 새로 나타남 (과거 FOV 내에 없었음)
- **disappeared** — 과거 시점에만 존재했고 현재 FOV 내에서 사라짐
- **past_not_included** — 현재 객체가 과거 이미지의 FOV 밖에 있어 비교 불가
- **current_not_included** — 과거 객체가 현재 이미지의 FOV 밖에 있어 비교 불가

각 페어링 레코드는 상태·과거·현재 정보(위경도·클래스·신뢰도·bbox·타임스탬프)와 함께 Pairing DB에 세션 단위로 저장되어 GraphRAG 인덱싱과 판독보고서 생성의 팩트 소스가 된다.

4. GraphRAG 기반 시공간 이력 누적

도 3. GraphRAG 기반 시공간 이력 누적 및 압축 컨텍스트 생성

페어링 결과를 AI 호출 없이 지식 그래프에 누적하고, 관련 이력을 압축해 LLM에 전달

[1단계] Pairing DB로부터 최근 회차 비교의 결과 배치를 조회 (한 회차 = 두 시점 영상 1쌍의 변화 탐지 결과)

예: 3회차 비교(2025.04.10 ↔ 08.20)에서 나온 페어링 레코드 배치

회차	클래스	상태	과거 좌표	현재 좌표	신뢰도
3회차	tank	matched	(37.5765, 126.9680)	(37.5765, 126.9680)	0.91
3회차	APC	new	-	(37.5771, 126.9698)	0.79
3회차	building	disappeared	(37.5750, 126.9660)	-	0.87

[2단계] 지도 + 약 1km 격자



같은 격자 셀 (약 1 km × 1 km)

격자화
(소수점 2자리)

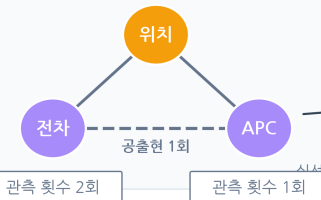
loc:37.58,126.97
asset:tank:37.58,126.97
asset:APC:37.58,126.97

← 위치 노드
← (전차 × 격자) 노드
← (APC × 격자) 노드

결정론적 노드 키 생성
결정론적 노드 키 (같은 격자 → 같은 키)

[3단계] Before (1~2회차 누적)

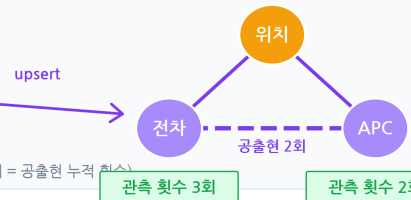
전차·APC 이미 같은 격자 관측 이력 존재



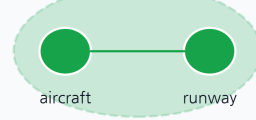
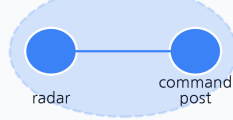
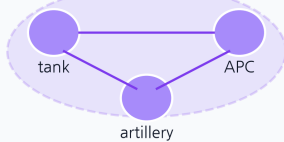
3회차 upsert (지금)
2025.04.10 ↔ 08.20
전차 · APC 같은 격자 재관측
→ 관측 횟수 +1, 공출현 엣지 +1

After (3회차 반영)

관측 횟수와 공출현 엣지 가중치 모두 +1



[4단계] Louvain 군집화 — 자주 함께 등장하는 자산들을 자동 그룹화



→ 자주 함께 등장한 자산들을 그래프 알고리즘이 자동으로 그룹화 (AI 호출 없음)
→ 각 군집은 별도 그래프가 아닌 graph_communities 레코드 (label + member_ids 리스트) 로 저장됨

[5단계] 지식 그래프 영속 저장

Graph DB (노드 + 엣지 + 군집)

[6단계] 보고서 생성 시 관련

질의 좌표 반경 R 내 조회

· 위치 노드(loc:*) 필터
· found_at 엣지로 연결된 asset 노드 수집
· 관측 횟수·상태별 카운터 (new/matched/disappeared)

Global Search

관련 커뮤니티 요약 조회

· Local 결과의 asset id들이 소속된 graph_communities 레코드를 조회
· label + member_summary 반환 (반복 관측 패턴)

압축 컨텍스트

≈ 500 토큰

"기갑 복합체 8회 반복 관측, 지난 30일간 3회 재배치, 반경 5km 내 지속 배치..."

→ LLM 프롬프트에 prepend

[도 3] GraphRAG 인덱싱 및 압축 컨텍스트 생성

4. GraphRAG 기반 시공간 이력 누적 (설명)

도 3은 매 회차 페어링 결과를 어떻게 시간축으로 쌓아 재활용 가능한 지식으로 만드는지를 6단계로 보여준다. 본 시스템의 지식 그래프는 위치별로 분리된 여러 개의 그래프가 아니라, 위치 노드와 자산 노드가 함께 존재하는 하나의 통합 그래프이다. 여기에 결정론적 노드 키 · LLM 호출 없는 upsert · Louvain 커뮤니티 탐지 · Local + Global 이중 검색이 결합되어 시공간 이력을 재활용 가능한 컨텍스트로 압축한다.

(1) 결정론적 노드 키와 통합 그래프 구조

위경도를 소수점 2자리로 반올림(약 1 km 격자)하여 결정론적 노드 키를 만든다. 두 종류의 노드가 존재한다:

- 위치 노드: loc:37.58,126.97 — (격자 셀)당 노드 1개
- 자산 노드: asset:tank:37.58,126.97 — (자산 클래스 × 격자)당 노드 1개

같은 격자에 여러 자산이 있으면 위치 노드 1개와 여러 개의 자산 노드가 함께 존재하며, 같은 클래스가 여러 격자에 있으면 격자 수만큼 별개 자산 노드가 생성된다. 두 노드 종류가 하나의 통합 그래프 안에서 공존하며, 위치별 서브그래프를 별도로 관리하지 않는다.

(2) 결정론적 upsert (LLM 비호출)

회차별 페어링 결과가 그래프 인덱서에 들어오면 다음 세 종류의 upsert가 순차 실행된다:

- 위치 노드 upsert — 같은 격자 키가 이미 있으면 관측 횟수 +1, 없으면 신규 삽입
- 자산 노드 upsert — (자산 × 격자) 키 기준으로 observation_count · 상태별 카운터 (new_count · matched_count · disappeared_count) · total_confidence · sessions 리스트 갱신
- 엣지 upsert — found_at(자산→위치)의 count·타임스탬프 갱신, co_occurred_with(자산↔자산, 같은 격자·같은 회차에 함께 관측된 조합)의 가중치와 locations 리스트 갱신

이 과정에는 LLM 호출이 전혀 없다. 순수 결정론적 함수(반올림 · 문자열 조합 · 사전 갱신)만으로 이루어지므로 동일 입력에 대해 항상 동일한 그래프가 재현되며, API 비용도 발생하지 않는다. 이는 GraphRAG 특유의 '재현 가능성'을 확보하는 결정적 요소다.

4. GraphRAG 기반 시공간 이력 누적 (설명 계속)

(3) Louvain 커뮤니티 탐지 — 반복 관측 패턴 자동 발견

누적된 통합 그래프에서 자산 노드들의 `co_occurred_with` 엣지 가중치를 대상으로 Louvain 알고리즘이 주기적으로 실행된다. Louvain은 그래프의 modularity를 최대화하는 방식으로 노드를 자동 분할하여, 여러 격자에 걸쳐 반복적으로 함께 관측되는 자산 조합을 하나의 커뮤니티로 묶는다. 예를 들어 여러 격자에서 tank·APC·artillery 세 클래스가 반복적으로 함께 관측되면, 각 격자별 asset 노드들이 하나의 커뮤니티에 소속된다.

중요한 점은 커뮤니티가 별도의 서브그래프로 저장되지 않는다는 것이다. 통합 그래프 구조는 그대로 두고, 각 커뮤니티는 `graph_communities` 테이블의 한 레코드로 저장된다:

컬럼	저장 값 예시
<code>community_index</code>	0
<code>label</code>	"Cluster-0: military tank, APC, artillery"
<code>member_ids (JSON)</code>	["asset:tank:37.58,126.97", "asset:APC:37.58,126.97", "asset:tank:37.60,127.01", "asset:APC:37.60,127.01", ...]
<code>member_summary</code>	결정론적 통계 요약 텍스트 (LLM 호출 없이 생성) — 예: "3개 격자에서 tank(관측 8회) · APC(6회) · artillery(4회) 공출현"
<code>summary</code>	선택적 LLM 요약 (기본 비활성)
<code>created_at</code>	커뮤니티 탐지 실행 시각

즉 각 커뮤니티는 "이 커뮤니티에 소속된 자산 노드 id 리스트"와 "통계 요약 텍스트"로만 저장되며, 별도의 서브그래프 순회 없이 레코드 조회로 빠르게 활용할 수 있다.

(4) Local Search — 대상 지역 자산 이력 조회

판독보고서 생성 시 우선 대상 지역 좌표를 중심으로 Local Search가 실행된다. 절차:

- ① 질의 좌표(lat_c, lon_c)와 반경 radius_deg(기본 0.05°, 약 5km)를 이용해 위치 노드(loc:*)를 필터링
- ② 각 위치 노드에서 found_at 엣지로 연결된 자산 노드를 수집
- ③ 자산 노드마다 관측 횟수 · 상태별 카운터(new/matched/disappeared/moved) · 평균 confidence · first_seen/last_seen 을 추출
- ④ 클래스별 총합 통계(예: tank 8회, APC 5회) 로 집계

출력 예: "이 격자 반경 5 km 내에 tank 8회, APC 5회, building 3회 관측. tank는 지난 30일간 3회 새로 등장, 1회 소실."

(5) Global Search — 관련 커뮤니티 요약 조회

Local Search로 얻은 자산 노드들이 어떤 반복 관측 패턴에 속하는지를 알아내기 위해 Global Search가 이어진다. 절차:

- ① Local Search 결과의 자산 노드 id들을 수집
- ② `graph_communities.member_ids`에 이 id들이 포함된 커뮤니티 레코드를 조회
- ③ 겹치는 id 수(local_overlap) 기준으로 정렬
- ④ 각 커뮤니티의 label + member_summary를 반환

출력 예: "이 지역 자산 조합은 'Cluster-0: tank, APC, artillery' 커뮤니티에 매핑됨 (local_overlap = 3). 이 커뮤니티는 전국 12개 격자에서 반복 관측되는 패턴."

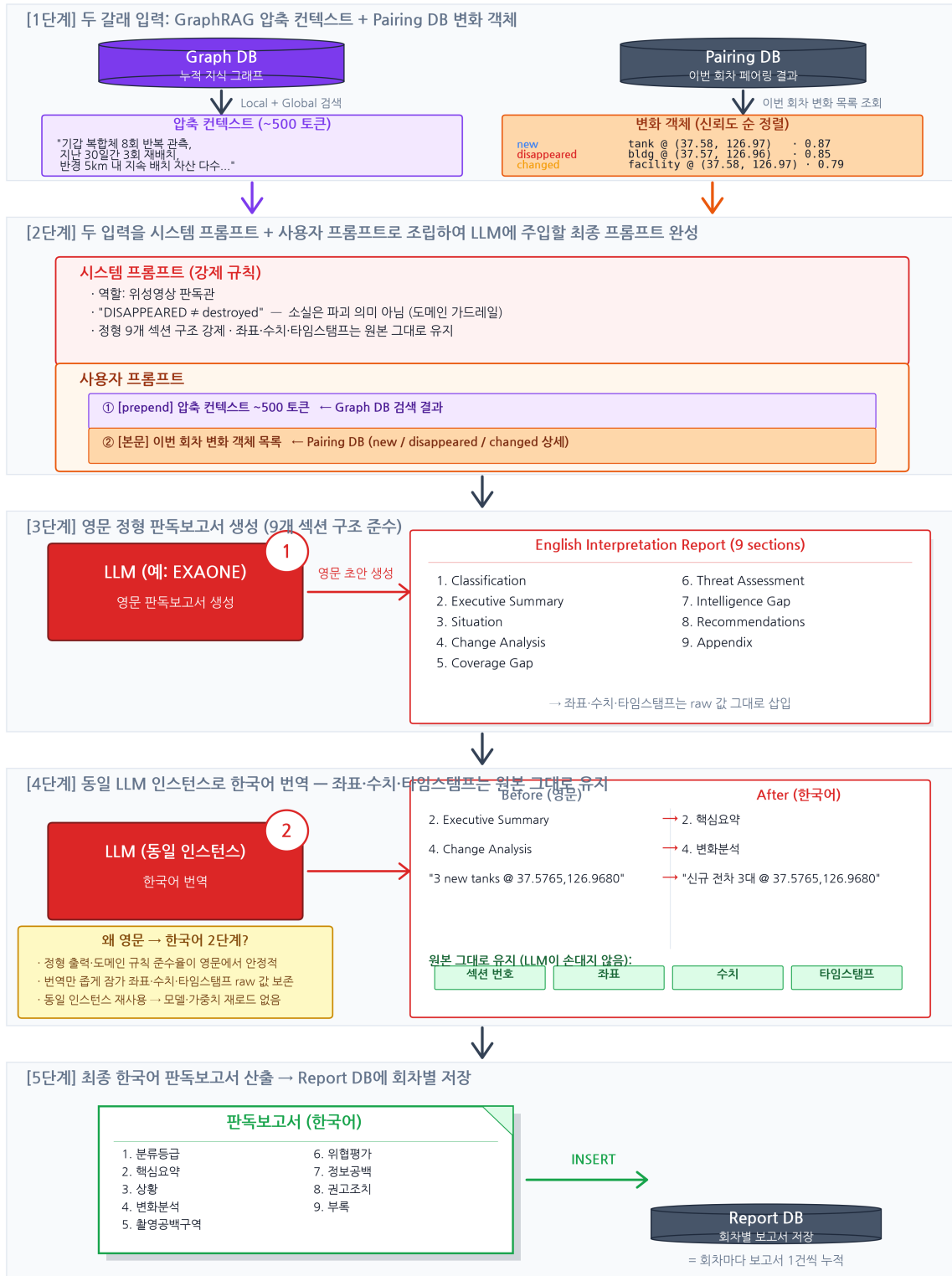
(6) 압축 컨텍스트 생성 · LLM 주입

Local + Global 결과를 결정론적 요약 규칙으로 결합해 **약 500 토큰의 압축 컨텍스트**를 만든다. 이 짧은 텍스트 블록이 판독보고서 생성 단계의 LLM 프롬프트 맨 앞에 **prepend** 되어, 이번 회차 변화 팩트(Pairing DB)와 함께 판독보고서의 시공간 맥락을 채운다. 압축이 결정론적이므로 같은 입력에 대해 항상 같은 컨텍스트가 재현되며, 그래프 전체를 프롬프트에 넣는 것 대비 토큰 소비가 크게 절감된다.

5. LangGraph Critic 기반 판독보고서 자율 생성

도 4. 판독보고서 자율 생성

그래프 압축 컨텍스트 + 이번 회차 변화 객체를 LLM에 주입 → 영문 정형 보고서 → 무오염 한국어 번역



[도 4] 판독보고서 생성 파이프라인

5. LangGraph Critic 기반 판독보고서 자율 생성 (설명)

도 4는 이번 회차 변화(팩트)와 과거 누적 맥락(배경)을 결합해 어떻게 완성도 있는 판독보고서를 만드는지를 보여준다. 두 입력의 역할이 다르다. **Pairing DB**는 이번 회차에 무엇이 변했는지의 팩트를 제공하고, **Graph DB**는 그 변화가 이례적인지 반복 패턴인지의 맥락을 제공한다. 두 소스를 시스템/사용자 프롬프트로 조립해 LLM에 넣으면 정형 9개 섹션(분류등급·핵심요약·상황·변화분석·촬영공백구역·위협평가·정보공백·권고조치·부록) 보고서가 생성된다.

Report Critic

LangGraph 파이프라인에서는 각 생성 단계 뒤에 **critic 노드**가 배치된다. **Report Critic**은 9개 섹션이 모두 존재하는지, 보고서에 인용된 자산·좌표가 실제 입력 페어링 레코드에 존재하는지, 'DISAPPEARED ≠ destroyed' 같은 도메인 가드레일이 준수되었는지를 검증한다. 검증 실패 시 실패 사유를 피드백으로 담아 재생성한다.

Translation Critic

이어 동일 LLM 인스턴스가 한국어로 번역하되 좌표·수치·타임스탬프는 원본 그대로 보존하며, **Translation Critic**이 섹션 구조·raw 값 보존·영문 잔재 여부를 검증한다. 두 단계의 critic 검증을 모두 통과한 최종 한국어 보고서가 Report DB에 세션별로 저장된다.

이 자기 검증 구조로 인해 환각 발생 가능성이 실질적으로 낮아지고, 좌표·수치 등 결정적 raw 값의 원본 보존이 보장된다. Critic이 여러 번 재시도해도 통과하지 못하는 케이스는 HITL로 자동 이관되어 사람이 최종 확인·수정할 수 있다.

6. LangGraph Critic 노드 아키텍처

전체 파이프라인은 LangGraph의 상태 머신으로 모델링된다. 각 생성 단계(generator 노드) 뒤에는 critic 노드가 배치되어 산출물의 품질을 자동 검증하며, 검증 결과에 따라 다음 단계로의 라우팅 · 같은 노드로의 재실행 · HITL(Human-in-the-Loop) 이관이 조건부로 결정된다. 총 5개의 critic 노드가 탐지 → 페어링 → 그래프 인덱싱 → 보고서 생성 → 번역 각 단계를 검증한다.

Critic 노드	검증 항목	실패 시 처리
① Detection Critic (SAM3 탐지 검증)	· 클래스·confidence 분포가 정상 범위인가 · degenerate bbox(0 크기)가 있는가 · 이미지 특성 대비 탐지 수가 비정상은 아닌가	SAM3 프롬프트 재조정 또는 confidence 임계값 완화 후 재실행. 반복 실패 시 HITL 이관
② Pairing Critic (이원 페어링 검증)	· matched : new : disappeared 비율의 급격한 편향 여부 · CLIP 유사도 분포가 임계값 근처에 몰려있진 않은가 · Gale-Shapley 후 미매칭 자산이 과다한가	SIMILARITY_MATCH_THRESHOLD 값을 조정하여 재실행. 임계값을 완화/강화해도 정상화되지 않으면 HITL 이관
③ Graph Critic (GraphRAG 인덱싱 검증)	· 격자 셀당 노드 수가 극단적으로 편중되지 않았는가 · Louvain 커뮤니티 개수·크기가 정상 범위인가 · upsert 후 관측 횟수·엣지 가중치가 실제로 반영됐는가	_LOC_PRECISION(격자 해상도)을 조정하여 재인덱싱. 경미한 편중은 경고만 남기고 통과 허용도 가능
④ Report Critic (영문 보고서 검증)	· 정형 9개 섹션 헤더가 모두 존재하는가 · 보고서에 인용된 자산·좌표·수치가 실제 입력 페어링 레코드에 존재하는가 (팩트 매칭) · 'DISAPPEARED ≠ destroyed' 등 도메인 가드레일 준수 여부	실패 사유를 프롬프트 피드백에 추가하여 재생성 (self-refine 패턴). 반복 실패 시 HITL 이관
⑤ Translation Critic (한국어 번역 검증)	· 9 섹션 번호·헤더가 그대로 유지됐는가 · 좌표·수치·타임스탬프가 raw 값 그대로 보존됐는가 (정규식 diff) · 영문 잔재가 없는가 (언어 일관성)	재번역 요청. 반복 실패 시 HITL 이관

파이프라인 상태(State) 관리

각 노드 간에는 LangGraph의 상태 객체(PipelineState)가 전달된다. 이 객체에는 세션 식별자, 각 단계의 산출물(detections · pairings · graph_context · english_report · korean_report), 각 critic의 통과/실패 이력(critic_history), 그리고 무한 루프 방지를 위한 단계별 재시도 카운터(retry_counts)가 포함된다.

재시도 정책과 HITL 이관

- 각 critic 노드는 최대 재시도 횟수(예: 3회)를 상태 객체로 관리하여 무한 루프를 방지한다
- 재시도가 소진되어도 검증에 통과하지 못하면 HITL 이관 노드로 라우팅되어 웹 UI를 통해 사람이 결과를 확인하고 수동 수정할 수 있다
- HITL에서 수정된 결과는 해당 session_id의 후속 DB 레코드를 자동 재계산하도록 다음 단계로 다시 흘러보낸다
- Critic은 대부분 규칙 기반(rule-based)으로 구현하여 결정론성과 재현성을 유지하고, LLM 기반 critic은 최소로 제한하여 GraphRAG의 결정론 강점을 훼손하지 않도록 설계한다

7. 데이터 아키텍처 (4개 DB)

각 파이프라인 단계의 산출물은 물리적으로 분리된 4개의 SQLite 데이터베이스에 세션 식별자(session_id)와 함께 저장된다. 이 구조는 (i) 결과의 재현·검증, (ii) 특정 단계의 부분 재실행, (iii) HITL 재처리 시 후속 단계 자동 재계산을 가능하게 한다.

7.1 Sensor DB — 촬영 영상과 SAM3 탐지 결과

image_records (이미지 메타)

컬럼	타입	저장 내용
id	UUID	이미지 고유 식별자
capture_time	DateTime	원본 촬영 시각
source_type	String	"satellite" 또는 "drone"
image_path	Text	이미지 파일 경로
lat_center / lon_center	Float	이미지 중심 좌표
lat/lon_min, lat/lon_max	Float	지리 bbox (픽셀→위경도 변환 기준)
resolution_m	Float	GSD (미터/픽셀)
sensor_platform	String	예: WorldView-3, MQ-9
det_width / det_height	Integer	탐지에 사용된 해상도
session_id	UUID	파이프라인 세션 식별자

detection_records (SAM3 탐지 결과)

컬럼	타입	저장 내용
id / image_id	UUID	탐지 식별자, 부모 이미지 FK
detection_time	DateTime	탐지 시각 (=capture_time)
object_class, object_class_index	String, Int	클래스명 및 인덱스
confidence	Float	SAM3 confidence
bbox_x1 / y1 / x2 / y2	Float	픽셀 단위 bbox
lat / lon	Float	픽셀→위경도 변환 결과
mask_rle	Text	RLE 인코딩된 SAM3 마스크
mask_area_px	Float	마스크 픽셀 수
source_type / extra / session_id	String, JSON, UUID	소스·부가정보·세션

7.2 Pairing DB — 두 시점 페어링 결과

각 회차 비교(과거 프레임 vs 현재 프레임)의 페어링 결과를 저장한다. 상태별로 채워지는 필드가 다르다: matched/changed는 양쪽 다, new는 current만, disappeared는 past만, past_not_included/current_not_included는 한쪽만 채워진다.

컬럼	타입	저장 내용
id, pairing_time	UUID, DateTime	레코드 식별자-페어링 시각
lat_center, lon_center	Float	지역 중심 좌표
status	String	matched changed new disappeared past_not_included current_not_included
current_detection_id	UUID	현재 탐지 FK (Sensor DB)
current_object_class, current_confidence	String, Float	현재 클래스·신뢰도
current_lat, current_lon	Float	현재 위경도
current_capture_time, current_bbox	DateTime, JSON	현재 시각·bbox
past_detection_id	UUID	과거 탐지 FK (Sensor DB)
past_object_class, past_confidence	String, Float	과거 클래스·신뢰도
past_lat, past_lon	Float	과거 위경도
past_capture_time, past_bbox	DateTime, JSON	과거 시각·bbox
source_type, session_id	String, UUID	소스·세션 식별자

7.3 Graph DB — 결정론적 지식 그래프

위경도 격자 반올림으로 만든 결정론적 노드 키를 사용해 자산·위치·관계를 upsert 방식으로 누적한다. AI 호출 없이 동일 입력에 동일 그래프가 재현되며, Louvain 알고리즘으로 반복 관측 패턴이 커뮤니티 레코드로 저장된다.

graph_entities (노드)

컬럼	타입	저장 내용
id (PK)	String	결정론적 노드 키 (loc:37.58,126.97 · asset:tank:37.58,126.97)
entity_type	String	location asset
name	String	표시 이름
properties	JSON	location: {lat, lon} · asset: {object_class, lat, lon, new_count, matched_count, changed_count, disappeared_count, moved_count, total_confidence, sessions}
first_seen / last_seen	DateTime	최초·최근 관측 시각
observation_count	Integer	누적 관측 횟수

graph_relations (엣지)

컬럼	타입	저장 내용
id	UUID	엣지 식별자
source_id / target_id	String	연결된 entity id
relation_type	String	found_at (자산→위치) co_occurred_with (자산↔자산)
properties	JSON	found_at: {count, confidence_sum, first_seen, last_seen, ...} · co_occurred_with: {count, locations}
created_at / updated_at	DateTime	생성·갱신 시각

graph_communities (Louvain 군집)

컬럼	타입	저장 내용
id, community_index	UUID, Int	커뮤니티 식별자·번호
label	String	예: Cluster-0: military tank, APC, artillery
member_ids	JSON	소속 자산 노드 id 리스트
member_summary	Text	결정론적 통계 요약 (LLM 호출 없이 생성)
summary	Text	선택적 LLM 요약
created_at	DateTime	생성 시각

7.4 Report DB — 최종 판독보고서

critic 노드 검증을 통과한 최종 한국어 판독보고서를 세션별로 저장한다.

컬럼	타입	저장 내용
id, saved_time	UUID, DateTime	레코드 식별자·DB 저장 시각
report_time	DateTime	LLM 생성 완료 시각
session_id	UUID	파이프라인 세션 식별자
llm_model	String	예: EXAONE-Deep-32B
llm_backend	String	vllm huggingface ollama
pairing_count	Integer	분석에 쓰인 페어링 레코드 수
file_path	Text	.txt 파일 경로 (옵션)
report_content	Text	한국어 9섹션 판독보고서 전문 (분류등급·핵심요약·상황·변화분석·촬영공백구역·위협평가·정보공백·권고조치·부록)

공통 특성

- **세션 격리**: 모든 테이블의 session_id로 세션 단위 조회·삭제·재실행
- **노드 키의 결정론성**: 같은 (자산, 격자) 조합이면 항상 같은 id → upsert 병합 자동
- **재현·검증 가능**: 각 단계 결과를 독립 조회할 수 있어 결과의 재현과 검증이 가능
- **부분 재계산**: HITL로 특정 단계 결과가 수정되면 후속 DB의 해당 session_id 레코드가 재계산

8. 사용 기술 스택

영역	사용 기술	역할
객체 탐지	SAM3 (Segment Anything Model 3)	사용자 지정 클래스의 zero-shot 인스턴스 탐지 (bbox + 마스크)
좌표 변환	pixel_to_geo() 함수	이미지 지리 bbox 선형 보간으로 픽셀 → 위경도
시각 임베딩	CLIP (ViT 기반)	마스크 배경 제거 후 crop → 코사인 유사도 비교용 임베딩
매칭 알고리즘	Gale-Shapley 안정 매칭	이동형 객체의 N×M 유사도 행렬에서 1:1 안정 매칭
지식 그래프	SQLAlchemy + NetworkX	결정론적 노드 키 기반 upsert · Louvain 커뮤니티 탐지
Agent 오케스트레이션	LangGraph	각 단계 노드 + critic 노드 · 조건부 라우팅 · 자기 검증 파이프라인
LLM	EXAONE (LG AI Research)	정형 9섹션 판독보고서 영문 생성 및 한국어 번역
LLM 서빙	vLLM	FastAPI와 동일 프로세스에서 EXAONE 모델 서빙
Backend	FastAPI + Uvicorn (Python)	REST API 서버 · 파이프라인 실행 · HITL 재처리 엔드포인트
Frontend	Vanilla JavaScript + HTML	React·Vue·Streamlit 미사용 · 경량 브라우저 UI
지도 시각화	Leaflet.js + OpenStreetMap	탐지 결과·페어링 상태의 지도 오버레이 시각화
데이터 저장	SQLite (4개 DB 분리)	Sensor / Pairing / Graph / Report DB · session_id 기반 격리

맺음말

본 프로젝트는 위성·드론 시계열 영상 분석에서 **정확한 변화 탐지와 맥락 있는 자율 판독**이라는 두 축을 하나의 end-to-end 파이프라인으로 통합하려는 시도이다. 특히 LangGraph critic 노드 기반의 자기 검증 구조를 도입함으로써 LLM 기반 시스템의 고질적 문제인 환각 위험을 자동으로 억제하고, 결정론적 GraphRAG 이력 누적을 통해 세션이 반복될수록 시스템의 판독 품질이 축적되도록 설계하였다. 이러한 자기 검증형 자율 판독 파이프라인은 위성 정보 판독·감시 경찰·재난 대응 등 실시간성과 신뢰성이 동시에 요구되는 분야에 폭넓게 적용될 수 있다.